Goroutines et Channels

Objectifs

1. Exécuter des tâches concurrentes avec des **Goroutines** et les synchroniser proprement sans jamais utiliser `time.Sleep`.
2. Choisir et configurer des **Channels non-bufferisés** (rendez-vous synchrone) et **bufferisés** (file FIFO asynchrone).
3. Sécuriser vos signatures de fonctions avec des **Channels directionnels** (`chan<- T` et `<-chan T`) et gérer la fermeture de flux (`close` + `for range`).
4. Diagnostiquer et corriger les blocages infinis (*Deadlocks*) détectés par le runtime Go.

Exercice 1 : Sonde de latence parallèle (*Fan-Out / Fan-In*)

Vous développez un outil de diagnostic réseau qui doit vérifier l'état de 5 serveurs distants. Exécuter ces vérifications de manière séquentielle prendrait la somme de toutes les latences, alors qu'en les lançant dans des Goroutines concurrentes, le temps total sera réduit à la latence du serveur le plus lent.

1. Définissez une structure `ResultatPing` contenant :
   - `Serveur` (`string`)
   - `Latence` (`time.Duration`)
   - `EnLigne` (`bool`)
2. Écrivez une fonction `verifierServeur(serveur string, dureeSimulee time.Duration, out chan<- ResultatPing)` :
   - Notez le type directionnel **`chan<- ResultatPing`** (*Send-Only*) qui interdit par compilation à cette fonction de lire dans le channel.
   - La fonction attend `time.Sleep(dureeSimulee)`, puis envoie une structure `ResultatPing` dans le channel `out` (considérez que le serveur est `EnLigne: true` si `dureeSimulee < 300 * time.Millisecond`, sinon `false`).
3. Dans `main()` :
   - Déclarez une map ou une liste de 5 serveurs avec leurs latences simulées :
     - `"api.paiement.internal"` : `120 ms`
     - `"db.replica-01.internal"` : `45 ms`
     - `"auth.sso.internal"` : `350 ms`
     - `"cache.redis.internal"` : `15 ms`
     - `"search.elastic.internal"` : `210 ms`

- Créez un channel `resultats := make(chan ResultatPing, 5)` (bufferisé à la taille du lot) et enregistrez l'heure de départ `debut := time.Now()`.
- Lancez les 5 appels à `verifierServeur` en parallèle avec le mot-clé **`go`**.
- Récupérez exactement les 5 résultats en lisant sur `<-resultats` dans une boucle classique `for i := 0; i < 5; i++`.

4. Affichez chaque résultat au fur et à mesure qu'il arrive, puis affichez le temps total d'exécution (`time.Since(debut)`).
    - Dans quel ordre les résultats s'affichent-ils ?
    - Pourquoi le temps total d'exécution est-il d'environ **`350 ms`** au lieu de **`740 ms`** (somme des latences) ?

Exercice 2 : Architecture en Pipeline à 3 étages et fermeture (`close`)

Le pattern **Pipeline** est l'une des architectures concurrentes les plus puissantes en Go : chaque étape du traitement tourne dans sa propre Goroutine, reçoit ses données d'un channel d'entrée (`<-chan`), les transforme, les pousse dans un channel de sortie (`chan<-`), puis **ferme** son channel de sortie lorsqu'elle a terminé.

Vous devez construire un pipeline de traitement de transactions financières en 3 étages : `[1. Générateur] ===(chan int)===> [2. Filtre & Taxe] ===(chan float64)===> [3. Agrégateur (main)]`

1. **Étage 1 — Le Générateur (`genererMontantsBruts`) :**
    - Signature : `func genererMontantsBruts(montants []int, out chan<- int)`
    - Parcourt la slice `montants`, envoie chaque valeur dans `out`, puis **ferme obligatoirement le channel `out`** avec `close(out)` avant de se terminer.
2. **Étage 2 — Le Processeur de filtrage et conversion (`appliquerTaxeEtFiltrer`) :**
    - Signature : `func appliquerTaxeEtFiltrer(in <-chan int, out chan<- float64)`
    - Utilise une boucle **`for m := range in`** pour lire les montants bruts jusqu'à la fermeture de `in`.
    - Ignore les montants négatifs ou nuls (`m <= 0`).
    - Pour les montants valides, applique une taxe de 20 % (`float64(m) * 1.20`) et envoie le résultat dans `out`.
    - Une fois la boucle `for range` terminée, **ferme obligatoirement le channel `out`** avec `close(out)`.
3. **Étage 3 — L'Agrégateur final (dans `main`) :**

- Créez les deux channels : `chBruts := make(chan int)` (non-bufferisé) et `chTaxes := make(chan float64)` (non-bufferisé).
- Lancez l'étage 1 et l'étage 2 dans deux Goroutines :

```Go
donnees := []int{100, -50, 250, 0, 400, -10, 50}
go genererMontantsBruts(donnees, chBruts)
go appliquerTaxeEtFiltrer(chBruts, chTaxes)
```

- Dans `main()`, consommez `chTaxes` avec une boucle `for montantTTC := range chTaxes`, calculez la somme totale TTC et affichez-la.

Exercice 3 : Chasse aux *Deadlocks* et aux *Panics* sur les Channels

Un développeur junior a écrit les trois extraits de code suivants qui plantent tous en production.

Analysez chacun des 3 cas, expliquez précisément la cause mécanique du plantage en vous appuyant sur la **Matrice de vérité des Channels**, puis écrivez une version corrigée dans votre code :

Cas A : Le rendez-vous manqué

```Go
func casA() {
	ch := make(chan string)
	ch <- "Message urgent" // Plantage ici : fatal error: all goroutines are asleep - deadlock!
	fmt.Println(<-ch)
}
```

- *Question :* Quelles sont les **deux manières différentes** de corriger ce code (l'une en gardant un channel non-bufferisé, l'autre sans lancer de nouvelle goroutine) ?

Cas B : Le consommateur infini

```Go
func casB() {
	ch := make(chan int, 3)
	ch <- 10
	ch <- 20
	ch <- 30
```

```
	// Le développeur veut lire toutes les valeurs présentes avec range :
	for v := range ch {
		fmt.Println(v) // Affiche 10, 20, 30 puis : fatal error: all goroutines are asleep - deadlock!
	}
}
```

- *Question :* Pourquoi `for v := range ch` bloque-t-il après avoir lu `30` alors que le channel est vide ? Quelle instruction unique manque-t-il avant la boucle ?

Cas C : Fermeture prématurée du mauvais côté

```go
func casC() {
	ch := make(chan int)
	go func() {
		for i := 1; i <= 5; i++ {
			ch <- i
		}
	}()
	fmt.Println("Premier élément lu :", <-ch)
	close(ch) // Le lecteur décide qu'il n'a plus besoin du reste et ferme le channel !
	time.Sleep(50 * time.Millisecond)
}
```

- *Question :* Quel crash critique (`panic`) se produit dans la goroutine productrice lorsque `main()` appelle `close(ch)` ? Quelle règle d'or sur la propriété du channel a été violée ici ?